Modern Testing Strategy & Prototype

Split Payment across Cards and Wallets

Course: SOFT411 — Software Validation & Testing
Institution: Final International University
Instructor: Asst. Prof. Dr. Ibrahim Adeshola
Assignment: Lecture 1 — *Principles of Software Testing*, slide 29
Student: Buğra Kor
Submission date: 3 October 2026

Deliverable checklist

Assignment requirement	Delivered	Where
A small web/mobile/API system or a clearly defined feature	FastAPI split payment service, ~400 lines	`app/`
Short risk analysis	10 risks scored by likelihood × impact × reversibility	Technical report §2
Functional + non-functional quality criteria	9 functional (F1–F9), 7 non-functional (N1–N7)	Technical report §3
At least **12** test cases using test design techniques	**23 designed**, 10 techniques, all automated	Test case design document
At least **4** automated checks	**34 executed pytest cases, all passing**	`tests/`
One CI/CD workflow that runs on change	GitHub Actions, 5 stages, blocking gates	`.github/workflows/ci.yml`
One security / performance / accessibility check	**Security** (Bandit SAST + 5 abuse cases) **and performance** (p95 budget + k6)	Security triage document, `perf/`
500–700 word reflection	**629 words**	Reflection document
Evidence: repository, test report, screenshots	Repository link, HTML + JUnit reports, Bandit report, performance JSON, defect-injection JSON, 6 screenshots	Evidence section, `reports/`

Results at a glance

Measurement	Result
Automated test cases	**34 passed, 0 failed** (1.11 s)
Static security scan (Bandit)	**0 findings** at any severity after documented triage
Performance smoke	mean 0.60 ms, **p95 0.77 ms** against a 15 ms budget
Defect injection (mutation-style)	**3 of 3 injected defects caught — 100 %**
Risks covered by blocking tests	R1, R2, R3, R6, R7, R8
Risks consciously deferred to monitoring	R4 (partly), R5, R9, R10

> **Note on the results above:** "34 of 34 passed" is the least informative number in this report. Section 6.1 documents the **seven mistakes** made while building the suite — including one case where the test was wrong and the application was right — because that is where the learning is.

Contents

1. Repository, test report and evidence — repository link, result summary, artefact index, screenshots
2. Technical report — scope, risk analysis, quality criteria, strategy, CI/CD, results, **what went wrong (§6.1)**, release recommendation
3. Test case design and traceability — 23 cases mapped to techniques and risks, plus defect-injection evidence
4. Security check and finding triage — SAST policy, abuse cases, explicit gaps
5. Reflection — what automation missed and where human judgement was necessary

Repository, Test Report and Evidence

1. Repository

Repository: https://github.com/REPLACE-ME/soft411-split-payment

> Replace the line above with the real URL once the repository is pushed, then rebuild this document with `tools/build_submission.py`. It is the only place in the submission where the URL appears.

Everything in this report can be reproduced from a clean checkout:

```
git clone <repository-url> && cd soft411-split-payment
python -m venv .venv
.venv/Scripts/python.exe -m pip install -r requirements.txt

.venv/Scripts/python.exe -m pytest --html=reports/test-report.html --self-contained-html
.venv/Scripts/python.exe -m bandit -r app -c bandit.yaml --severity-level medium
.venv/Scripts/python.exe tools/defect_injection.py
```

The same three commands are the three stages of the CI workflow, so a reviewer running them locally sees exactly what the pipeline gates on.

2. Test execution summary

Recorded run — `reports/junit.xml`, 3 October 2026, Python 3.11.9 on Windows 10:

Measurement	Result	Source artefact
Automated test cases executed	**34 passed, 0 failed, 0 skipped, 0 errors**	`reports/junit.xml`, `reports/test-report.html`
Suite wall-clock duration	1.11 s (931 ms in tests)	`reports/pytest-console.txt`
Designed test cases	23 (TC01–TC23), all automated	Test case design document
Test design techniques applied	10	Test case design document
Static security scan (Bandit)	**0 findings** at medium and high severity, 401 lines scanned	`reports/bandit.txt`, `reports/bandit-full.txt`
Abuse-case security tests	5 passed	`reports/junit.xml` (`test_security.py`)
Performance smoke (p95 latency)	**0.77 ms** against a 15 ms budget, 200 requests	`reports/performance-smoke.json`
Defect injection (mutation-style)	**3 of 3 injected defects caught — 100 %**	`reports/defect-injection.json`

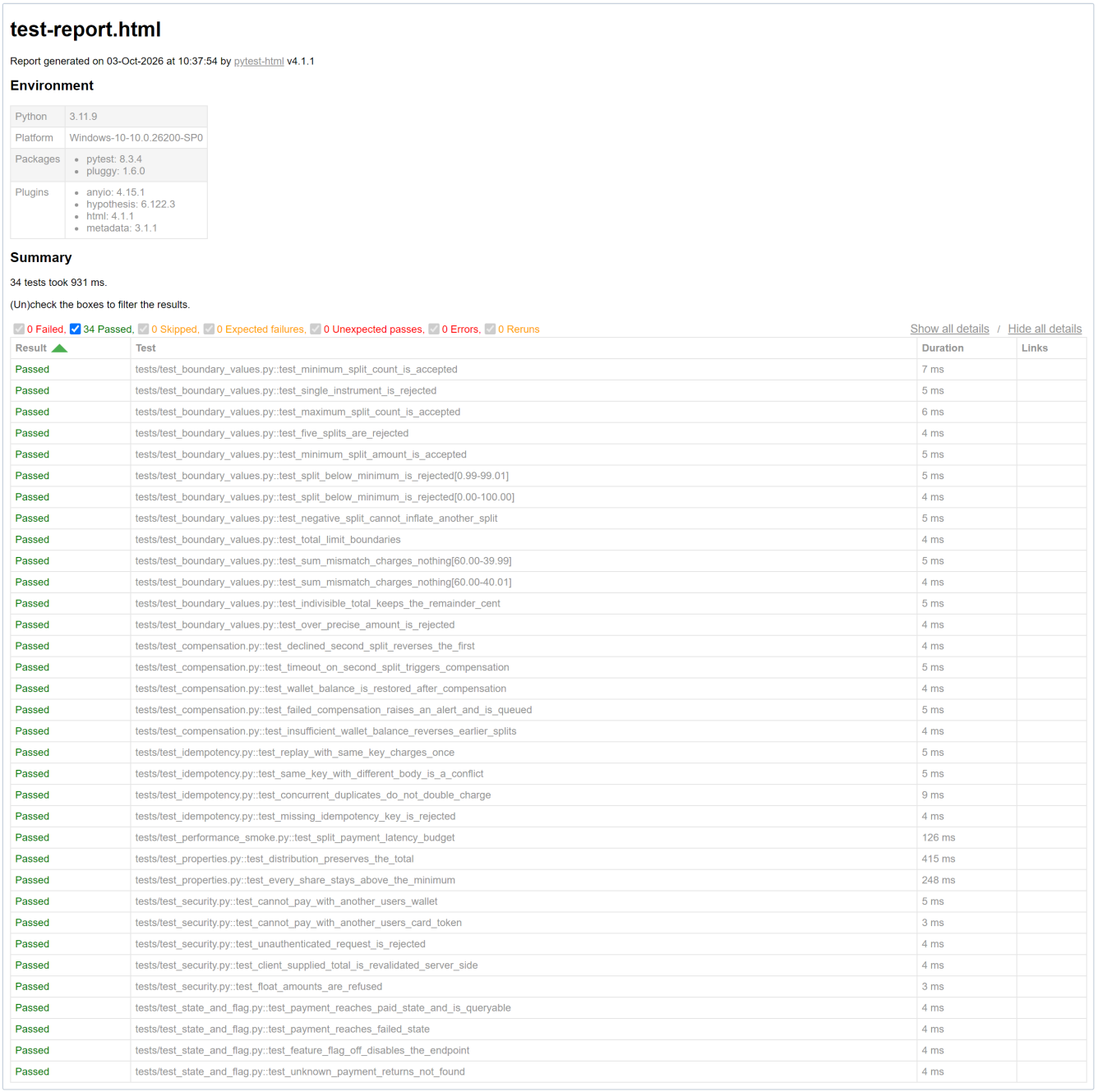
Breakdown of the 34 executed cases by test file:

Test file	Cases	What it covers
`test_boundary_values.py`	13	split count, amount and total boundaries, cent distribution
`test_compensation.py`	5	decline, timeout and insufficient-balance rollback paths
`test_idempotency.py`	4	replay, body-fingerprint conflict, concurrency, missing key
`test_security.py`	5	ownership, authentication, server-side revalidation, float refusal
`test_state_and_flag.py`	4	paid/failed state transitions, feature-flag rollback, 404
`test_properties.py`	2	property-based invariants (500 generated examples)
`test_performance_smoke.py`	1	p95 latency budget

3. Submitted artefacts

Artefact	File	What it proves
HTML test report	`reports/test-report.html`	per-case results with durations and environment
JUnit XML	`reports/junit.xml`	machine-readable result totals, consumed by the CI summary
Bandit report	`reports/bandit.txt`, `reports/bandit-full.txt`	security scan at the gating severity and at every severity
Performance measurement	`reports/performance-smoke.json`	latency percentiles against a declared budget
Defect-injection result	`reports/defect-injection.json`	the suite fails when the application is broken
Console transcript	`reports/pytest-console.txt`	the raw run output behind the numbers above
Screenshots	`reports/screenshots/`	the figures below

4. Screenshots



test-report.html

Report generated on 03-Oct-2026 at 10:37:54 by pytest-html v4.1.1

Environment

Python	3.11.9
Platform	Windows-10-10.0.26200-SP0
Packages	• pytest: 8.3.4 • pluggy: 1.6.0
Plugins	• anyio: 4.15.1 • hypothesis: 6.122.3 • html: 4.1.1 • metadata: 3.1.1

Summary

34 tests took 931 ms.

(Un)check the boxes to filter the results.

0 Failed, 34 Passed, 0 Skipped, 0 Expected failures, 0 Unexpected passes, 0 Errors, 0 Reruns — Show all details / Hide all details

Result	Test	Duration	Links
Passed	tests/test_boundary_values.py::test_minimum_split_count_is_accepted	7 ms	
Passed	tests/test_boundary_values.py::test_single_instrument_is_rejected	5 ms	
Passed	tests/test_boundary_values.py::test_maximum_split_count_is_accepted	6 ms	
Passed	tests/test_boundary_values.py::test_five_splits_are_rejected	4 ms	
Passed	tests/test_boundary_values.py::test_minimum_split_amount_is_accepted	5 ms	
Passed	tests/test_boundary_values.py::test_split_below_minimum_is_rejected[0.99-99.01]	5 ms	
Passed	tests/test_boundary_values.py::test_split_below_minimum_is_rejected[0.00-100.00]	4 ms	
Passed	tests/test_boundary_values.py::test_negative_split_cannot_inflate_another_split	5 ms	
Passed	tests/test_boundary_values.py::test_total_limit_boundaries	4 ms	
Passed	tests/test_boundary_values.py::test_sum_mismatch_charges_nothing[60.00-39.99]	5 ms	
Passed	tests/test_boundary_values.py::test_sum_mismatch_charges_nothing[60.00-40.01]	4 ms	
Passed	tests/test_boundary_values.py::test_indivisible_total_keeps_the_remainder_cent	5 ms	
Passed	tests/test_boundary_values.py::test_over_precise_amount_is_rejected	4 ms	
Passed	tests/test_compensation.py::test_declined_second_split_reverses_the_first	4 ms	
Passed	tests/test_compensation.py::test_timeout_on_second_split_triggers_compensation	5 ms	
Passed	tests/test_compensation.py::test_wallet_balance_is_restored_after_compensation	4 ms	
Passed	tests/test_compensation.py::test_failed_compensation_raises_an_alert_and_is_queued	5 ms	
Passed	tests/test_compensation.py::test_insufficient_wallet_balance_reverses_earlier_splits	4 ms	
Passed	tests/test_idempotency.py::test_replay_with_same_key_charges_once	5 ms	
Passed	tests/test_idempotency.py::test_same_key_with_different_body_is_a_conflict	5 ms	
Passed	tests/test_idempotency.py::test_concurrent_duplicates_do_not_double_charge	9 ms	
Passed	tests/test_idempotency.py::test_missing_idempotency_key_is_rejected	4 ms	
Passed	tests/test_performance_smoke.py::test_split_payment_latency_budget	126 ms	
Passed	tests/test_properties.py::test_distribution_preserves_the_total	415 ms	
Passed	tests/test_properties.py::test_every_share_stays_above_the_minimum	248 ms	
Passed	tests/test_security.py::test_cannot_pay_with_another_users_wallet	5 ms	
Passed	tests/test_security.py::test_cannot_pay_with_another_users_card_token	3 ms	
Passed	tests/test_security.py::test_unauthenticated_request_is_rejected	4 ms	
Passed	tests/test_security.py::test_client_supplied_total_is_revalidated_server_side	4 ms	
Passed	tests/test_security.py::test_float_amounts_are_refused	3 ms	
Passed	tests/test_state_and_flag.py::test_payment_reaches_paid_state_and_is_queryable	4 ms	
Passed	tests/test_state_and_flag.py::test_payment_reaches_failed_state	4 ms	
Passed	tests/test_state_and_flag.py::test_feature_flag_off_disables_the_endpoint	4 ms	
Passed	tests/test_state_and_flag.py::test_unknown_payment_returns_not_found	4 ms	

Figure 1 — pytest HTML report. All 34 cases Passed, 0 Failed. Test names are readable, which is deliberate: each one states the rule it protects, so the report doubles as a specification.

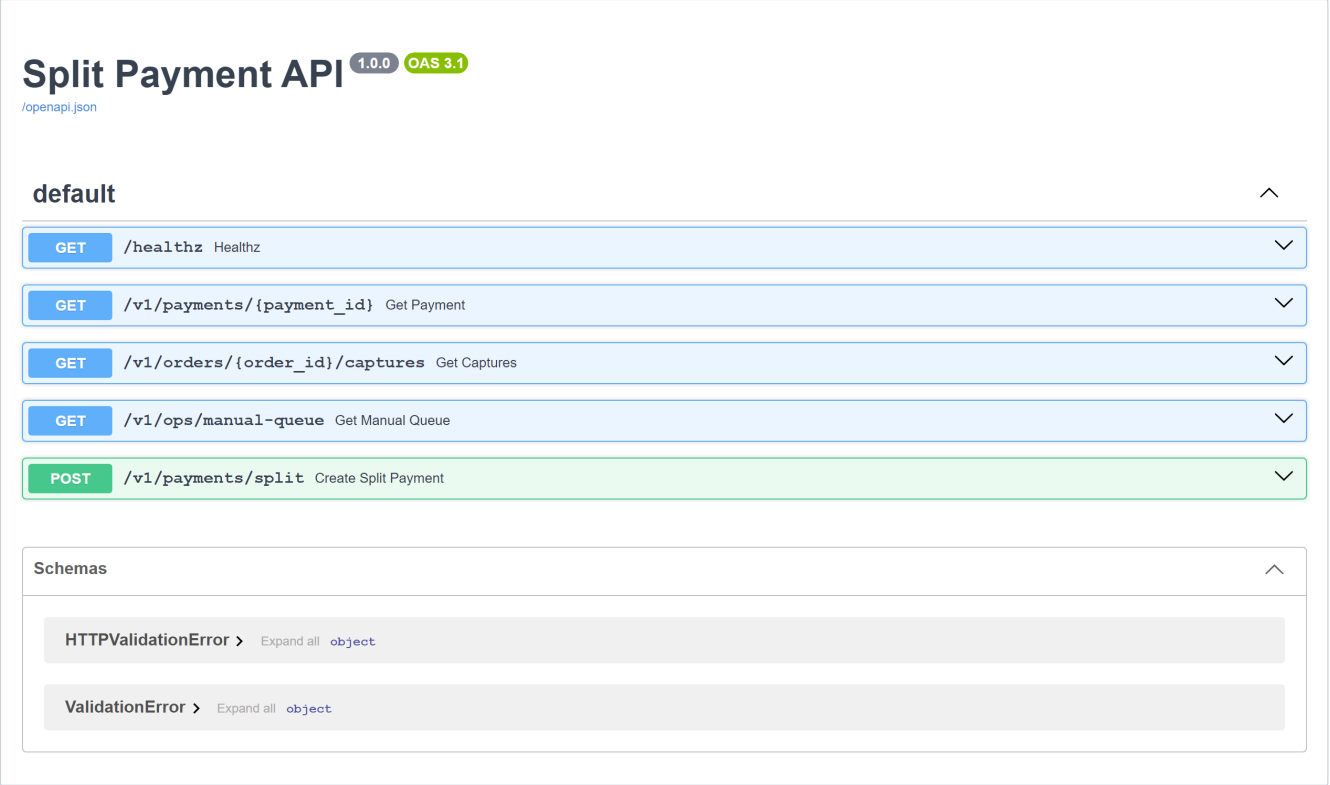


Figure 2 — the system under test. The generated OpenAPI surface: `POST /v1/payments/split` plus the audit endpoints that make "no double charge" assertable (`/v1/orders/{order_id}/captures`) and the manual-intervention queue used when compensation itself fails.

```
pytest — 34 passed

$ pytest --html=reports/test-report.html --self-contained-html

.............................                                    [100%]
============================== warnings summary ===============================
.venv\Lib\site-packages\starlette\testclient.py:40
  C:\Users\Buğra\Desktop\software validation and testing\assignment\.venv\Lib\site-packages\starlette\testclient.py:40: DeprecationWarning: The
anyio.abc.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = typing.Callable[[], typing.ContextManager[anyio.abc.BlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
- generated xml file: C:\Users\Buğra\Desktop\software validation and testing\assignment\reports\junit.xml -
- Generated html report: file:///C:/Users/Bu%C4%9Fra/Desktop/software%20validation%20and%20testing/assignment/reports/test-report.html -
34 passed, 1 warning in 1.11s
```

Figure 3 — test run console output. `34 passed in 1.11 s` , with the JUnit and HTML artefacts written in the same run.

```
Bandit SAST — 0 findings

$ bandit -r app -c bandit.yaml --severity-level medium

Run started:2026-10-03 07:37:45.731281

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 401
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 4

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
Files skipped (0):
```

Figure 4 — Bandit static security scan. No issues identified across 401 lines. The four suppressed checks are documented and justified in the security triage section; they are suppressions with reasons, not a silenced scanner.

Defect injection — 3 of 3 caught

```
$ python tools/defect_injection.py

{
  "baseline": "34 passed, 0 failed in 1.36s",
  "restored": "34 passed, 0 failed in 1.19s",
  "mutants": [
    {
      "id": "M1",
      "risk": "R1 double charge",
      "description": "idempotency lookup always misses - replays are charged again",
      "caught": true,
      "summary": "31 passed, 3 failed in 1.34s",
      "failing_tests": [
        "tests/test_idempotency.py::test_replay_with_same_key_charges_once",
        "tests/test_idempotency.py::test_same_key_with_different_body_is_a_conflict",
        "tests/test_idempotency.py::test_concurrent_duplicates_do_not_double_charge"
      ]
    },
    {
      "id": "M2",
      "risk": "R2 incorrect totals",
      "description": "sum check removed - splits no longer have to match the total",
      "caught": true,
      "summary": "31 passed, 3 failed in 1.48s",
      "failing_tests": [
        "tests/test_boundary_values.py::test_sum_mismatch_charges_nothing[60.00-39.99]",
        "tests/test_boundary_values.py::test_sum_mismatch_charges_nothing[60.00-40.01]",
        "tests/test_security.py::test_client_supplied_total_is_revalidated_server_side"
      ]
    },
    {
      "id": "M3",
      "risk": "R3 partial charge",
      "description": "compensation disabled - captured splits stay captured",
      "caught": true,
      "summary": "29 passed, 5 failed in 1.32s",
      "failing_tests": [
        "tests/test_compensation.py::test_declined_second_split_reverses_the_first",
        "tests/test_compensation.py::test_timeout_on_second_split_triggers_compensation",
        "tests/test_compensation.py::test_wallet_balance_is_restored_after_compensation",
        "tests/test_compensation.py::test_failed_compensation_raises_an_alert_and_is_queued",
        "tests/test_compensation.py::test_insufficient_wallet_balance_reverses_earlier_splits"
      ]
    }
  ],
  "detection_rate_percent": 100.0
}
```

Figure 5 — defect injection. Three defects were injected one at a time and the suite re-run: broken idempotency, a removed sum check and disabled compensation. Each was caught by the tests that own the corresponding risk, and the baseline was restored afterwards. Detection rate 100 %.

Figure not captured yet — `06-ci-run.png`

Figure 6 — CI/CD quality gate. The GitHub Actions workflow executing on a push: security scan, then the blocking functional suite, then the performance budget, with the evidence uploaded as a build artefact.

5. How this evidence was produced

The figures are rendered from the committed artefacts rather than re-captured by hand (`tools/make_screenshots.py`), so the report and `reports/` cannot drift apart; the document itself is assembled by `tools/build_submission.py`. Figure 6 is the one exception, as a workflow run only exists on the hosting platform.

Modern Testing Strategy & Prototype — Technical Report

Course: SOFT411 Software Validation & Testing, Final International University
Assignment: Lecture 1 — *Principles of Software Testing*, slide 29
Feature under test: split payment — one order total paid with 2–4 instruments (cards and wallets) in a single request
Repository layout: `app/` service, `tests/` suite, `perf/` k6, `.github/workflows/` CI, `docs/`, `reports/`

---

1. Introduction and Scope

The chosen feature is the one from Case Study 1: a payment application lets a customer pay a single order with several instruments at once. It is small enough to build and test properly, and it concentrates the risks that make payment software hard — money must be counted exactly, requests must survive being retried, and a transaction that spans two external providers can fail halfway through.

A prototype service (~400 lines) was implemented so the tests would have something real to execute against, rather than being written in the abstract. Everything the service does is traceable to a risk; nothing was added for completeness.

In scope: amount validation and cent distribution, idempotency, sequential capture with compensation, instrument ownership, feature-flag rollback, latency budget.

Out of scope: card tokenisation, 3-D Secure, persistence in a real database, TLS and secret management, PCI-DSS scope assessment, any user interface.

Assumptions (the case statement gives no product rules; stating them is itself a static-testing activity, and ambiguity is a risk in its own right):

#	Assumption
A1	A split payment uses at least 2 and at most 4 instruments
A2	Each split is ≥ 1.00 and the order total ≤ 50 000.00
A3	Amounts have at most 2 decimals; `sum(splits)` must equal `total` to the cent
A4	Card network and wallet service are independent external providers
A5	Every payment request carries an `Idempotency-Key`
A6	The feature can be switched off by configuration without a deployment
A7	A void/refund mechanism exists for compensating a partial charge

---

2. Risk Analysis

Risk score = likelihood × impact (1–5 each). Impact weighs financial loss, regulatory exposure and, decisively, **reversibility**.

ID	Risk	How it occurs	L	I	Score	Reversible?
R1	**Double charge**	user double-clicks, client retries, network drops, webhook delivered twice	4	5	**20**	No — customer money is taken
R2	**Incorrect totals**	float rounding, undistributed remainder cent, tampered total	4	5	**20**	No — accounting breaks
R3	**Partial charge**	split 1 captured, split 2 fails, compensation does not run	4	5	**20**	Partly — needs void/refund
R4	**Timeout / latency**	several provider calls in one request; p95 budget breached	5	3	**15**	Yes — retry and monitoring
R5	Contract breakage	wallet provider changes schema or error codes	3	4	12	Yes — contract tests
R6	**Regression**	the new path breaks existing single-instrument payments	3	5	**15**	No — affects all traffic
R7	**Authorisation flaw**	paying with another user's wallet or leaked token	2	5	**10**	No — security incident
R8	Race condition	parallel duplicates consume the same balance twice	3	4	12	Partly
R9	Rare combinations	mixed currencies, exotic card types	2	2	4	Yes
R10	Accessibility / UI defect	unclear error, keyboard and screen-reader gaps	3	1	3	Yes

R1, R2 and R3 share the top score, and all three are irreversible — that is why they drive the strategy. R4 is the most *likely* risk but its impact is recoverable, which is exactly why it is handled with a threshold and monitoring rather than a hard gate.

---

3. Quality Criteria

3.1 Functional criteria (must hold for every request)

#	Criterion	Verified by
F1	2–4 splits; anything else is rejected before any capture	TC01–TC04
F2	Each split ≥ 1.00; zero, negative and over-precise amounts rejected	TC05–TC07, TC10b
F3	Order total ≤ 50 000.00	TC08

#	Criterion	Verified by
F4	`sum(splits) == total` to the cent, and an indivisible total loses no cent	TC09, TC10, TC22
F5	**Atomicity:** either all splits are captured, or none remain captured	TC09, TC14–TC16b
F6	**Idempotency:** a replayed request never charges twice	TC11–TC13b
F7	Every instrument must belong to the authenticated caller	TC17–TC19
F8	A failed compensation is queued for a human and raises an alert	TC16
F9	Payment state is observable afterwards (`PAID` / `FAILED` with cause)	TC20, TC20b

3.2 Non-functional criteria

#	Attribute	Target	Verified by
N1	Performance	p95 < 15 ms application latency; p95 < 2.5 s end-to-end in a real environment	`test_performance_smoke.py` , `perf/k6-split-smoke.js`
N2	Security	no medium/high SAST finding; all abuse cases rejected with no money moved	Bandit, TC17–TC19b
N3	Reliability	no provider failure leaves money captured; failed compensation is never silent	TC14–TC16b
N4	Observability	captures are queryable per order; manual queue and alerts are inspectable	audit endpoints used by the tests
N5	Operability	the feature can be switched off in under a minute without a deployment	TC21
N6	Maintainability	the suite runs in ~1 s so it can gate every commit	CI timing
N7	Accessibility	**not assessed** — the prototype has no UI	documented gap

4. Test Strategy

The strategy follows the lecture's risk-based reasoning: effort is proportional to *irreversibility*, not to the size of the component.

Level choice. Almost all checks sit at the unit and API level. Money arithmetic is verified in isolation (fast, exhaustive-ish via property-based testing), and behaviour is verified through the HTTP boundary where the rules actually live. There is no end-to-end UI layer because there is no UI; in the full product this is where three targeted Playwright journeys would sit.

Techniques applied (detail in `docs/TEST-CASES.md`): boundary-value analysis, equivalence partitioning, decision rules, state-transition testing, concurrency testing, fault injection, abuse-case testing, property-based testing and a threshold-gated performance smoke test.

Oracles are side effects, not status codes. Every negative case asserts through the audit endpoint that zero captures exist. A `403` that still moved money would satisfy a status-only assertion, so the assertion was designed around the invariant instead.

What is deliberately not automated. R9 (rare multi-currency combinations) and R10 (accessibility) have no automated coverage. They are low impact, cheap to detect in production, and expensive to simulate; they are accepted behind a feature flag with monitoring. Recording that decision is part of the strategy — an untested area that nobody wrote down is a blind spot, while one that is written down is a managed risk.

5. Automation and the CI/CD Pipeline

`.github/workflows/ci.yml` runs on every push, pull request and manual dispatch:

Stage	Step	Gate
1	Bandit SAST on `app/`	fails on medium/high severity
2	`pytest -m "not performance"` — boundary, idempotency, compensation, security, state	**blocking**, any failure stops the pipeline
3	`pytest -m performance` — p95 latency budget	**blocking** on threshold breach
4	Upload `test-report.html` , `junit.xml` , `bandit.txt` , `performance-smoke.json`	evidence artefact, runs even on failure
5	Write the pass/fail counts into the GitHub step summary	visible without opening artefacts

Fast checks run first so feedback arrives in seconds. The whole gate completes in a few seconds locally, which is what makes it acceptable on every commit — a gate that is slow or flaky gets bypassed, and then it protects nothing.

6. Results and Evidence

All artefacts are committed under `reports/` .

Evidence	Result	File
Functional and security suite	**34 passed, 0 failed** in 1.11 s	`junit.xml` , `test-report.html` , `pytest-console.txt`
Static security scan	**0 findings** at any severity after triage (4 low-severity false positives suppressed per line with justification)	`bandit.txt` , `bandit-full.txt` , `docs/SECURITY-TRIAGE.md`

Evidence	Result	File
Performance smoke	mean 0.60 ms, p50 0.58 ms, **p95 0.77 ms** vs a 15 ms budget	`performance-smoke.json`
Defect injection	**3 of 3 injected defects caught — 100 %**	`defect-injection.json`

The defect-injection result is the most informative number here. A green suite only proves that the tests passed; injecting a real defect and watching the right tests fail is what demonstrates defect-finding power:

Mutant	Injected defect	Outcome
M1	idempotency lookup always misses	caught — TC11, TC12, TC13 fail
M2	`sum(splits) == total` check removed	caught — TC09 (both variants), TC19 fail
M3	compensation disabled	caught — TC14, TC15, TC15b, TC16, TC16b fail

---

6.1 What Went Wrong While Testing

The green suite is the least informative part of this project. Seven mistakes were made and corrected along the way; none of them was an application crash, and every one of them was about the **quality of the evidence** the suite produces. They are recorded here because they are the part with transferable value.

#	Mistake	Root cause	Correction	Lesson
1	The concurrency case failed, and the **test** was wrong — it demanded `409` from all seven losers of the race, while the service returned one `201` and seven `200` replays	The assertion was tied to status codes, but rejecting an in-flight duplicate and replaying the stored response are equally safe	Asserted the invariant instead: exactly two captures per order; status may be `200` or `409` and a replay must return the same `payment_id`	Automation executes an oracle, it does not supply one. No tool could say which expectation was correct
2	All 34 checks passed on the first complete run, which was reassuring and uninformative	The suite had only ever run against correct code, and a weak assertion passes exactly like a strong one	Wrote `tools/defect_injection.py` and injected three defects matching R1, R2 and R3	A passing suite is a claim; a defect that the right tests catch is evidence
3	The first injected defect failed 16 tests across unrelated files instead of the 3 it targeted	The patch left the idempotency record in an in-flight state, so a later call raised `KeyError` — a crash was injected, not the defect under study	Moved the mutation one level down, into the lookup in `store.begin()`	A defect experiment is only useful if the blast radius is small enough to read
4	The injection script reported an empty summary, and once mistook a `FAILED ...` line for the result	pytest's console output was being scraped with a regex; line order on Windows was not what the script assumed	Read `reports/junit.xml` instead — the machine-readable report pytest already writes	If a tool offers machine-readable output, measure from that, not from text written for humans
5	The performance threshold was set at 60 ms while the measured p95 was 0.77 ms, so the check could never fail	The number was chosen before any measurement, "to be safe"	Measured first, then set the budget at 15 ms (~15× the observed value)	A threshold is a sensitivity/flakiness trade-off and cannot be chosen honestly without data
6	Bandit raised four false "hardcoded password" findings, and the first suppression comment made it emit warnings on every run	The strings are fault-injection selectors in the mock; the justification had been written on the same line as `# nosec`, so Bandit parsed it as rule names	Per-line `# nosec B105` with the reason on the preceding line, triage written up in `docs/SECURITY-TRIAGE.md`, CI gate set to medium severity	Skipping the rule project-wide would also hide a real secret committed later; triage means deciding with a reason you can show
7	Installing the dependencies globally downgraded `starlette` / `uvicorn` and broke an unrelated project on the same machine; the CI file also carried a flag for a pytest plugin that was never installed	`pip install` was run without an isolated environment, and a command line was copied without checking it against `requirements.txt`	Created a project `.venv`, restored the global versions, pinned every dependency, removed the invalid flag	Test results are only as trustworthy as the environment: an unpinned shared environment means green locally and red in CI with no way to say which was right

Grouped by what they have in common:

- **The oracle is human** (1, 5) — a tool can run a check, but deciding the correct answer and an honest threshold stayed a human responsibility.
- **Signal beats drama** (2, 3, 4) — a green suite, a 16-test failure and a scraped log line all look like information and carry almost none.
- **Context is part of the test** (6, 7) — a finding needs triage and an environment needs pinning, or the result is neither reproducible nor trustworthy.

---

7. Residual Risk and Release Recommendation

Risk	Status	Mitigation
R1, R2, R3, R6, R7, R8	covered by blocking tests	gate on every commit
R4	partially covered	application-level budget in CI; k6 against staging; p95/timeout dashboard and alerts in production
R5	**not covered** — the providers are mocked	consumer-driven contract tests (Pact) before a real integration
R9, R10	**not covered** by design	flag-disabled at first, segment error rates monitored, accessibility audit scheduled when a UI exists

Recommendation: *conditional go* — release behind `SPLIT_PAYMENT_ENABLED` with a 1 % canary, automatic rollback if the payment success rate drops more than 2 % below baseline or p95 exceeds 3 s, and an hourly reconciliation job watching for duplicate captures with zero tolerance.

Mapping to the lecture's principles: the evidence above reports residual risk rather than claiming correctness (principle 1), coverage was chosen by risk because exhaustive testing is impossible (principles 2 and 4), the cheapest checks run first and earliest (principle 3), property-based testing and defect injection keep the

suite from ageing into a ritual (principle 5), the irreversibility of money defines what blocks a release in this domain (principle 6), and the untested accessibility and usability gaps are recorded because a technically correct payment can still fail its users (principle 7).

The 629-word reflection in `docs/REFLECTION.md` discusses what automation missed and where human judgement remained necessary.

Test Case Design and Traceability

Feature under test: **split payment** — one order total paid with 2–4 instruments (cards and wallets) in a single request.

The assignment asks for at least 12 designed test cases. 23 are designed below and **all 23 are automated**, which expands to 34 executed pytest cases because four of them are parameterised. Every case states the design technique it comes from and the risk it covers, so a reviewer can see *why* the case exists rather than only *what* it does.

Risk IDs refer to the risk analysis in `docs/TECHNICAL-REPORT` (R1 double charge, R2 incorrect totals, R3 partial charge, R4 timeout/latency, R6 regression, R7 authorisation, R8 race condition).

TC	Title	Technique	Risk	Input	Expected result	Automated test
TC01	Minimum split count accepted	BVA — lower valid boundary	R2	2 instruments, 60.00 + 40.00 = 100.00	`201 PAID`, 2 captures	`test_boundary_values.py::test_minimum_split_cou`
TC02	Single instrument rejected	BVA — invalid below boundary	R2	1 instrument, 100.00	`400 VALIDATION_ERROR`, 0 captures	`::test_single_instrument_is_rejected`
TC03	Maximum split count accepted	BVA — upper valid boundary	R2	4 × 25.00	`201 PAID`, 4 captures, total 100.00	`::test_maximum_split_count_is_accepted`
TC04	Five splits rejected	BVA — invalid above boundary	R2	5 × 20.00	`400 TOO_MANY_SPLITS`, 0 captures	`::test_five_splits_are_rejected`
TC05	Minimum split amount accepted	BVA	R2	99.00 + 1.00	`201 PAID`, total 100.00	`::test_minimum_split_amount_is_accepted`
TC06	Split below minimum rejected	BVA + equivalence partitioning	R2	0.99 and 0.00 (2 variants)	`400 VALIDATION_ERROR`, 0 captures	`::test_split_below_minimum_is_rejected`
TC07	Negative split rejected	EP — invalid class	R2, R7	105.00 + (−5.00)	`400 VALIDATION_ERROR`, 0 captures	`::test_negative_split_cannot_inflate_another_sp`
TC08	Order total limit boundaries	BVA — both sides of the limit	R2	50 000.00 and 50 000.01	`201` then `400 AMOUNT_LIMIT_EXCEEDED`	`::test_total_limit_boundaries`
TC09	Sum mismatch charges nothing	Decision rule + atomicity	R2, R3	60.00 + 39.99 / 60.00 + 40.01	`400 SPLIT_SUM_MISMATCH`, 0 captures	`::test_sum_mismatch_charges_nothing`
TC10	Indivisible total keeps the remainder cent	BVA — rounding boundary	R2	33.34 + 33.33 + 33.33	`201`, captured total exactly 100.00	`::test_indivisible_total_keeps_the_remainder_ce`
TC10b	Over-precise amount rejected	EP — invalid class	R2	66.666666	`400 VALIDATION_ERROR`	`::test_over_precise_amount_is_rejected`
TC11	Replay with the same key charges once	State-based (idempotency)	**R1**	same key + same body, twice	`201` then `200`, same `payment_id`, still 2 captures	`test_idempotency.py::test_replay_with_same_key_`
TC12	Same key, different body is a conflict	State-based, negative	**R1**	same key, different order	`409 IDEMPOTENCY_KEY_REUSE`, 0 captures	`::test_same_key_with_different_body_is_a_confli`
TC13	Concurrent duplicates do not double charge	Concurrency test	**R1, R8**	8 parallel identical requests	exactly one `201`; others `200` / `409`; 2 captures	`::test_concurrent_duplicates_do_not_double_char`
TC13b	Missing idempotency key rejected	Negative test	R1	no `Idempotency-Key` header	`400 IDEMPOTENCY_KEY_REQUIRED`	`::test_missing_idempotency_key_is_rejected`
TC14	Declined second split reverses the first	Fault injection	**R3**	split 2 declined	`402 FAILED`, compensation `COMPLETED`, 0 captures	`test_compensation.py::test_declined_second_spli`
TC15	Timeout triggers compensation	Fault injection	**R3, R4**	split 2 times out	`402 TIMEOUT`, 0 captures	`::test_timeout_on_second_split_triggers_compens`
TC15b	Wallet balance restored	State verification	R3	provider error after a wallet capture	wallet balance equals the pre-payment value	`::test_wallet_balance_is_restored_after_compens`
TC16	Failed compensation alerts and queues	Fault injection — error path of the error path	R3	void itself fails	`MANUAL_INTERVENTION_REQUIRED`, 1 queue item, alert raised	`::test_failed_compensation_raises_an_alert_and_`

TC	Title	Technique	Risk	Input	Expected result	Automated test
TC16b	Insufficient balance on the last split	Fault injection	R3	4th split exceeds the wallet balance	`402 INSUFFICIENT_FUNDS`, 0 captures	`::test_insufficient_wallet_balance_reverses_ear`
TC17	Another user's wallet cannot be used	Abuse case — OWASP API1 (BOLA)	**R7**	wallet owned by `user_2`	`403 FORBIDDEN_INSTRUMENT`, 0 captures	`test_security.py::test_cannot_pay_with_another_`
TC17b	Another user's card token cannot be used	Abuse case — API1	R7	leaked card token	`403 FORBIDDEN_INSTRUMENT`	`::test_cannot_pay_with_another_users_card_toker`
TC18	Unauthenticated request rejected	Negative security test	R7	no identity header	`401 UNAUTHENTICATED`, no state created	`::test_unauthenticated_request_is_rejected`
TC19	Client-supplied total re-validated	Abuse case — client-trusted input	R2, R7	total 10.00, splits sum 100.00	`400 SPLIT_SUM_MISMATCH`	`::test_client_supplied_total_is_revalidated_ser`
TC19b	JSON float amounts refused	Defensive input typing	R2	`60.0` as a JSON number	`400 VALIDATION_ERROR`	`::test_float_amounts_are_refused`
TC20	Payment reaches PAID and is queryable	State-transition testing	R6	happy path then `GET /v1/payments/{id}`	stored state `PAID`, total and splits match	`test_state_and_flag.py::test_payment_reaches_pa`
TC20b	Payment reaches FAILED	State-transition testing	R6	declined split	stored state `FAILED` with cause	`::test_payment_reaches_failed_state`
TC21	Feature flag off disables the endpoint	Configuration test — rollback control	R6	`SPLIT_PAYMENT_ENABLED=false`	`404 FEATURE_DISABLED`, `/healthz` reports it	`::test_feature_flag_off_disables_the_endpoint`
TC21b	Unknown payment returns 404	Negative path	—	unknown id	`404 NOT_FOUND`	`::test_unknown_payment_returns_not_found`
TC22	Distribution preserves the total	**Property-based** (Hypothesis, 300 examples)	R2	random totals × 2–4 parts	`sum(shares) == total`, max 2 decimals, balanced	`test_properties.py::test_distribution_preserves`
TC22b	Every share stays above the minimum	Property-based (200 examples)	R2	random totals × 2–4 parts	`min(shares) >= 1.00`	`::test_every_share_stays_above_the_minimum`
TC23	Latency budget (performance check)	Performance smoke with a threshold	**R4**	200 sequential split payments	p95 < 15 ms, mean < 10 ms	`test_performance_smoke.py::test_split_payment_l`

Coverage by technique

Technique	Cases
Boundary-value analysis	TC01–TC06, TC08, TC10
Equivalence partitioning	TC02, TC06, TC07, TC10b, TC19b
Decision rules / atomicity	TC09, TC14–TC16b
State-transition testing	TC11–TC13b, TC20, TC20b
Concurrency testing	TC13
Fault injection	TC14–TC16b
Abuse-case (security) testing	TC17–TC19b
Property-based testing	TC22, TC22b
Performance testing	TC23
Configuration / rollback testing	TC21

Coverage by risk

Risk	Covered by	Blocking?
R1 double charge	TC11, TC12, TC13, TC13b	Yes
R2 incorrect totals	TC01–TC10b, TC19, TC19b, TC22, TC22b	Yes
R3 partial charge	TC09, TC14, TC15, TC15b, TC16, TC16b	Yes

Risk	Covered by	Blocking?
R4 timeout / latency	TC15, TC23 (+ `perf/k6-split-smoke.js`)	Threshold only
R6 regression	TC20, TC20b, TC21, whole suite in CI	Yes
R7 authorisation	TC07, TC17, TC17b, TC18, TC19	Yes
R8 race condition	TC13	Yes
R9 rare combinations, R10 accessibility	**not covered** — monitored after deployment	No

The last row is deliberate, not an omission: see the reflection for why those two risks are accepted with monitoring instead of tests.

Defect-injection evidence

A green suite only proves the tests passed. `tools/defect_injection.py` injects three known defects one at a time and re-runs the suite:

Mutant	Injected defect	Result
M1	Idempotency lookup always misses (replays are charged again)	**Caught** — 3 tests fail (TC11, TC12, TC13)
M2	Sum check removed (splits need not match the total)	**Caught** — 3 tests fail (TC09 ×2, TC19)
M3	Compensation disabled (captured splits stay captured)	**Caught** — 5 tests fail (TC14–TC16b)

Detection rate: **100 %** — recorded in `reports/defect-injection.json`.

Security Check and Finding Triage

The assignment requires at least one security, performance or accessibility check. This project includes **two security layers** plus a performance check.

1. Static analysis (SAST) — Bandit

```
bandit -r app -c bandit.yaml -f txt -o reports/bandit.txt --severity-level medium
```

Gate policy: the CI step fails on *medium* and *high* severity findings. Low severity findings are still written to the report and triaged here. A gate that blocks on known false positives is abandoned by the team within a week, which is worse than a gate that is honest about what it enforces.

Findings and triage

ID	Finding	Location	Severity	Decision
B105 ×4	"Possible hardcoded password" on `tok_declined`, `tok_timeout`, `tok_provider_error`, `tok_void_fails`	`app/psp.py`	Low (Medium confidence)	**False positive — suppressed per line with `# nosec B105`.** These string literals are fault-injection selectors inside the *mock* provider, not credentials. A real PSP client would hold no tokens in code.
B101 (assert_used)	n/a	—	—	Rule disabled in `bandit.yaml`. Asserts exist only in the test suite, which is excluded from the scan; the application uses explicit error responses.

Result after triage: **0 findings at any severity** (`reports/bandit.txt`).

Suppressions are per line and carry a justification. A blanket `--skip B105` across the project would also hide a genuinely hardcoded secret introduced later.

2. Dynamic abuse-case tests

Static analysis cannot see broken authorisation, so five executable abuse cases cover the OWASP API Security Top 10 items that matter for a payment endpoint:

Test	OWASP item	What it proves
TC17 `test_cannot_pay_with_another_users_wallet`	API1 — Broken Object Level Authorisation	A wallet belonging to `user_2` cannot be charged by `user_1`
TC17b `test_cannot_pay_with_another_users_card_token`	API1	A leaked card token is useless from another account
TC18 `test_unauthenticated_request_is_rejected`	API2 — Broken Authentication	No identity, no payment, and no state is created
TC19 `test_client_supplied_total_is_revalidated_server_side`	API3 — Client-trusted input	A tampered `total` cannot make the caller pay less than the sum of the splits
TC19b `test_float_amounts_are_refused`	Defensive input typing	JSON floats cannot represent money exactly and are refused at the boundary

Each of these asserts the **side effect**, not only the status code: after a rejected request the audit endpoint must report zero captures. A `403` that still moved money would pass a status-only assertion.

3. What is deliberately not covered

Area	Why not here	How it is handled instead
Dependency vulnerabilities	Out of scope for a prototype with 7 pinned dependencies	`pip-audit` / Dependabot in a real pipeline
TLS, secret storage, key rotation	Infrastructure concern, no infrastructure in this prototype	Platform responsibility, reviewed at design time
Rate limiting / brute force	No authentication service in scope	Gateway-level control
PCI-DSS scope and card data storage	The prototype never stores card data — it holds opaque tokens only	Would require a formal audit, not a test suite
Accessibility (WCAG 2.2)	The prototype is API-only; there is no user interface to audit	Would need axe-core or Lighthouse against a real UI

Stating these gaps explicitly is part of the deliverable: a security report that lists only what was checked, without what was not, overstates the assurance it provides.

Reflection — What Automation Missed, and Where Human Judgement Was Still Necessary

Automating 34 checks for the split payment feature took far less effort than deciding *what* to automate and *what the result meant*. The suite is green and the defect injection shows it has real detection power, yet almost every decision that gave the suite its value came from human reasoning, not from a tool.

The clearest example appeared when the concurrency test failed. Eight parallel requests with the same idempotency key produced one `201` and seven `200` replays, while my test demanded `409` responses. The application was correct and the test was wrong. No tool could have told me that, because both outcomes are legitimate: rejecting a duplicate while the first request is in flight and replaying the stored response afterwards are equally safe. What mattered was the invariant underneath — exactly two captures against the order. Rewriting the assertion around that invariant instead of around status codes was a judgement about *what a double charge actually is*. Automation executes an oracle; it does not supply one.

A second limitation was that a passing suite proves almost nothing on its own. All 34 tests passed on the first complete run, which was reassuring and uninformative. I had to inject three defects deliberately — disabling the idempotency lookup, removing the sum check, and switching off compensation — to see whether the tests would notice. They did, and the three failure sets were precisely the ones targeting those risks. Choosing *those three* defects was again human work: they were selected because they map to the irreversible financial risks, not because a tool proposed them.

The performance check exposed how easily a number can be mistaken for assurance. My pytest smoke test reports a p95 around 0.8 ms, which looks excellent and means very little: it runs in-process against mocked providers, so it measures my own code and nothing else. Real split payments make several external calls whose latency dominates the response. The automated check is therefore useful only as a regression detector, and I set the threshold at 15 ms — roughly fifteen times the observed value — after weighing sensitivity against flakiness. A tighter budget would catch smaller regressions but fail randomly on slower CI machines, and a gate that fires randomly gets ignored. That trade-off is a judgement call with no correct answer.

Static analysis behaved similarly. Bandit produced four "hardcoded password" findings, all of them string comparisons inside the mock provider. Suppressing the rule globally would have been quicker, but it would also hide a genuine secret committed later, so I suppressed the four lines individually with written justifications and set the CI gate at medium severity. Reading a finding, deciding it is a false positive, and choosing a suppression that does not create a blind spot is triage, not automation.

Some risks resisted automation entirely. I could not test whether the extra cent in 100.00 ÷ 3 should go to the first split, the largest split, or the customer's own card — that is a product and accounting decision, and my test merely encodes the assumption I made. Equally, the suite says nothing about whether the error message after a declined split is understandable, whether rare multi-currency combinations behave sensibly, or whether the API would pass an accessibility review, because there is no interface to review. Those risks are accepted consciously and pushed to monitoring, behind a feature flag and a canary rollout, with reconciliation alerts as the detector.

The most important judgement was the release decision itself. Classifying double charges, incorrect totals and partial charges as blocking while treating latency and rare combinations as monitorable rests on reversibility: money taken from a customer cannot be un-taken by a dashboard. No coverage metric encodes that. Automation gave me fast, repeatable evidence; deciding which evidence was sufficient to ship remained a human responsibility.